

Software Requirements Specification

for

EVNet Sentinel: Anomaly Detection for EV Charging Infrastructure

Version 1.2

Prepared by:

Group 1

Mukta Varak 24102A0013
Shruti Chaurasiya 24102A0018
Shardul Chogale 24102A0021
Neha Chavhan 24102A0022

Vidyalankar Institute of Technology
July 28, 2026

Revision History

Name	Date	Reason For Changes	Version
Group 1	–	Initial Draft based on Project Specification	1.0
Group 1	–	Revised to IEEE 830/29148 standard: corrected dataset description against source papers, added quantitative/verifiable acceptance criteria to every requirement, added Requirements Traceability Matrix, added literature benchmark targets, added reproducibility & evaluation requirements, added related-work positioning for eventual publication.	1.1
Group 1	July 28, 2026	Migrated dashboard/UI stack from Streamlit to a decoupled Next.js/React frontend (with animation/interactive-simulation support) backed by a Python model-serving API; updated all External Interface, System Feature, and Non-functional requirements affected by this architecture change; updated DFD, TBD list, and Traceability Matrix accordingly.	1.2

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Document Conventions	1
1.3	Intended Audience and Reading Suggestions	1
1.4	Product Scope	1
1.5	References	2
2	Overall Description	3
2.1	Product Perspective	3
2.2	Product Functions	3
2.3	User Classes and Characteristics	3
2.4	Operating Environment	4
2.5	Design and Implementation Constraints	4
2.6	User Documentation	5
2.7	Assumptions and Dependencies	5
2.8	Positioning vs. Related Work	5
3	External Interface Requirements	6
3.1	User Interfaces	6
3.2	Hardware Interfaces	7
3.3	Software Interfaces	7
3.4	Communications Interfaces	7
4	System Features	8

4.1	Data Preprocessing & Feature Engineering Pipeline	8
4.1.1	Description and Priority	8
4.1.2	Stimulus/Response Sequences	8
4.1.3	Functional Requirements	8
4.2	ML Detection Engine (Static & Online)	8
4.2.1	Description and Priority	8
4.2.2	Functional Requirements	8
4.3	Attack Coverage Evaluation Module	9
4.3.1	Description and Priority	9
4.3.2	Functional Requirements	9
4.4	Alerting and Visualization Dashboard (Revised in v1.2)	9
4.4.1	Description and Priority	9
4.4.2	Functional Requirements	9
4.5	Evaluation & Reproducibility Requirements	10
4.5.1	Description and Priority	10
4.5.2	Functional Requirements	10
5	Other Nonfunctional Requirements	10
5.1	Performance Requirements	10
5.2	Safety Requirements	11
5.3	Security Requirements	11
5.4	Software Quality Attributes	11
5.5	Business Rules	12
6	Other Requirements	12
A	Appendix A: Glossary	13
B	Appendix B: Analysis Models	13
C	Appendix C: To Be Determined List	14
D	Appendix D: Requirements Traceability Matrix	14
E	Appendix E: Literature Benchmark Reference Table	15

1 Introduction

1.1 Purpose

This document specifies the software requirements for **EVNet Sentinel**, a machine-learning-based Intrusion Detection System (IDS) for Electric Vehicle Charging Stations (EVCS). The system ingests network-flow records derived from the public CICEVSE2024 dataset, engineers flow-level features, and trains/evaluates classical batch classifiers alongside an online (streaming) classifier with drift adaptation, to detect cyberattacks such as Denial-of-Service (DoS), reconnaissance, and host-level compromise directed at EV charging networks. Detection results are served to an interactive, animated Next.js/React dashboard through a Python model-serving API. The document is written to IEEE Std 830-1998 structure (extended with a traceability matrix, per IEEE/ISO/IEC 29148 practice) so the specification is auditable, testable, and reusable as the basis for a course report or a conference/journal submission.

1.2 Document Conventions

- **Terminology:** Technical terms (e.g., EVSE, CSMS, OCPP, ADWIN, ARF, SSR, WS) are defined in Appendix A.
- **Formatting:** Headings follow the IEEE 830-1998 structure. Requirements are labeled with unique identifiers (e.g., UI-1, REQ-1.1) for traceability; every functional requirement carries an explicit, measurable **Acceptance Criterion** so it can be verified rather than merely reviewed.
- **Priority:** All requirements are mandatory (*High*) unless explicitly marked *Medium* or *Low*.
- **Source tags:** Where a requirement or target value is grounded in a cited paper rather than derived from the course brief, the source is given in square brackets, e.g. [Makhmudov et al., 2025].

1.3 Intended Audience and Reading Suggestions

- **Development Team:** Implements the data pipeline, ML models, backend API, and Next.js dashboard. Focus on Section 3 (External Interfaces), Section 4 (System Features), and Appendix D (Traceability Matrix).
- **Security Analysts & Researchers:** Evaluate the model against the CICEVSE2024 dataset and compare against published baselines. Focus on Section 2, Section 4.6 (Evaluation & Reproducibility), and Appendix E (Benchmark Reference Table).
- **Academic Evaluators / Reviewers:** Assess the project against course outcomes and, where applicable, publication standards. Focus on Section 1, Section 2.9 (Positioning vs. Related Work), and Section 5.

1.4 Product Scope

EVNet Sentinel is an applied cybersecurity research prototype delivering:

- **Data Ingestion & Preprocessing:** Automated cleaning and feature engineering of the CICEVSE2024 network-traffic subset.
- **ML Detection Engine:** Training and comparative evaluation of static/batch models (Random Forest, SVM, Logistic Regression, Decision Tree) and an online-learning model (Adaptive Random Forest with ADWIN drift detection).
- **Attack Coverage Evaluation:** Evaluation against the attack traffic already captured in CICEVSE2024, supplemented by a controlled, clearly-labelled synthetic traffic generator used only to stress-test robustness beyond the recorded attack set.
- **Model-Serving API:** A Python backend service that runs the trained models and exposes them to the frontend over REST and WebSocket.
- **Interactive Alerting Dashboard:** A Next.js/React single-page application, rendering animated, near-real-time (simulated-stream) visualizations of metrics, confusion matrices, and anomalous events.

What this project explicitly delivers (in scope): a reproducible offline/simulated-stream ML pipeline; a comparative benchmark of five classifiers (four static + one online); a drift-adaptation demonstration; a decoupled backend API and an interactive animated dashboard; a CSV alert log; a written comparison of results against the literature baselines in Appendix E.

Exclusions (explicitly out of scope, unchanged from the original brief): real-world penetration testing of physical EVSE hardware; production deployment to a live CSMS or utility/grid network; hardware/firmware reverse engineering; use of the CICEVSE2024 *host-event* (HPC/kernel-event) or *power-consumption* data dimensions; live network-interface capture; public/production hosting of the dashboard or API (local demonstration only, see Section 2.5).

1.5 References

1. IEEE Std 830-1998: *IEEE Recommended Practice for Software Requirements Specifications*.
2. ISO/IEC/IEEE 29148:2018: *Systems and software engineering — Life cycle processes — Requirements engineering* (used here only for the traceability-matrix convention; the base document remains IEEE 830).
3. F. Makhmudov, D. Kilichev, U. Giyosov, F. Akhmedov, “Online Machine Learning for Intrusion Detection in Electric Vehicle Charging Systems,” *Mathematics*, vol. 13, no. 5, p. 712, 2025. <https://doi.org/10.3390/math13050712>
4. C. Joglekar, C. Eze, D. Xiang, A. Monti, “A Hybrid Intrusion Detection System for Electric Vehicle Charging Infrastructure,” *arXiv:2606.23236*, 2026. <https://arxiv.org/pdf/2606.23236>
5. J. Bean, D. M. Manias, “Cybersecurity of Electric Vehicle Charging Infrastructure: Recent Advances, Open Challenges, and Future Directions,” *arXiv:2605.24190*, 2026. <https://arxiv.org/abs/2605.24190>
6. E. D. Buedi, A. A. Ghorbani, S. Dadkhah, R. L. Ferreira, “Enhancing EV Charging Station Security Using a Multi-dimensional Dataset: CICEVSE2024,” in *Data*

and Applications Security and Privacy XXXVIII (DBSec 2024), LNCS vol. 14901, Springer, 2024, pp. 171–190. https://www.researchgate.net/publication/378869277_Enhancing_EV_Charging_Station_Security_Using_A_Multi-dimensional_Dataset_CICEVSE2024

7. Source Code Reference (Makhmudov et al. pipeline): https://github.com/TATU-hacker/Intrusion_Detection_on_Electric_Vehicle_Charging_Systems
8. CIC EV Charger Attack Dataset 2024 (CICEVSE2024), Canadian Institute for Cybersecurity, University of New Brunswick. <https://www.unb.ca/cic/datasets/evse-dataset-2024.html>

2 Overall Description

2.1 Product Perspective

EVNet Sentinel is a standalone software prototype composed of two decoupled tiers: (1) a Python **model-serving backend** (data pipeline, trained classifiers, REST/WebSocket API) and (2) a **Next.js/React frontend** that renders the dashboard and drives interactive/animated visualizations by consuming that API. This client-server split replaces the v1.1 monolithic Streamlit design, in which the UI and the ML runtime shared a single Python process. It acts as an analytical layer that would, in a production deployment, sit between the Electric Vehicle Supply Equipment (EVSE) and the Charging Station Management System (CSMS), consistent with the network-level vantage point used in [Makhmudov et al., 2025]. It operates independently of any live power grid and processes representative data *as if* deployed at the network edge; no live interface tapping occurs (see Section 2.5).

2.2 Product Functions

- **Data Parsing:** Converts CICEVSE2024 network-flow CSV records into ML-ready arrays.
- **Model Benchmarking:** Compares Random Forest, SVM, Logistic Regression, and Decision Tree models on accuracy, precision, recall, F1-score, and inference latency.
- **Concept Drift Handling:** Implements an Adaptive Random Forest (ARF) with ADWIN to detect and adapt to evolving attack patterns in a simulated data stream, replicating the approach validated in [Makhmudov et al., 2025].
- **Model Serving:** Exposes trained-model inference, metrics, and a live alert stream over a documented REST/WebSocket API.
- **Interactive Visualization:** Renders animated charts, transitioning confusion matrices, and a live alert feed in the browser, decoupled from the ML runtime.

2.3 User Classes and Characteristics

- **Administrator / Researcher:** Loads datasets, triggers training pipelines, tunes hyperparameters, and reviews classification reports.

- **Traffic Simulator (System Role):** Programmatically replays labelled CICEVSE2024 traffic and, optionally, injects supplementary synthetic malicious traffic to test the pipeline beyond the recorded attack set.
- **Security Analyst / Dashboard Viewer:** End-user observing the Next.js dashboard for alerts, investigating anomalies via interactive charts, and supporting human-in-the-loop decisions on suspected false positives.

2.4 Operating Environment

- **Platform:** Cross-platform (Windows, macOS, Linux) for both tiers.
- **Backend Software Stack:** Python 3.10+, scikit-learn, pandas, NumPy, River (online learning / ADWIN), plus a lightweight async API framework (**FastAPI**, recommended for native WebSocket + async support) to serve predictions, metrics, and the alert stream. (*Changed in v1.2 – Streamlit removed; Streamlit could not be kept because it is a self-contained Python-rendered app and cannot act as a pure JSON/WebSocket API for an external Next.js frontend.*)
- **Frontend Software Stack (New in v1.2):** Node.js 20+ LTS; **Next.js 14+** (**React 18+**, **TypeScript**); Tailwind CSS for styling; **Framer Motion** for interactive/animated transitions (alert pop-ins, state changes, simulation playback); a charting library compatible with animation, e.g. **Recharts** or **visx/D3.js**, for the metrics panel and confusion matrix; **socket.io-client** or the native **WebSocket** API for the live alert stream.
- **Hardware:** Standard student laptop (minimum 8 GB RAM, modern multi-core CPU); no GPU required.

2.5 Design and Implementation Constraints

- **Data Limitation:** The system is bounded by the static, already-captured nature of the CICEVSE2024 network-traffic subset (data corresponding to two physical/emulated chargers, EVSE-A and EVSE-B, per [Buedi et al., 2024]). Attack techniques not represented in this dataset, and not reproduced by the supplementary synthetic-traffic generator, cannot be detected.
- **Real-Time Simulation Constraints:** As this is an academic prototype, “near real-time” is simulated by streaming the static dataset sequentially (row-by-row or mini-batch) rather than tapping a live network interface. Latency figures reported by the system (Section 5.1) describe per-instance inference time, not end-to-end network latency.
- **Single-Modality Constraint:** CICEVSE2024 is multi-dimensional – it also contains host-event (HPC/kernel) and power-consumption data [Buedi et al., 2024], which a hybrid approach can fuse for improved accuracy [Joglekar et al., 2026]. This project deliberately uses only the network-traffic dimension; the multi-modal extension is recorded as future work in Section 2.9, not as a requirement.
- **Language/Runtime Boundary (New in v1.2):** scikit-learn and River are Python-only; they cannot execute inside the Next.js/Node.js runtime. Consequently a backend API process is now a *mandatory* architectural element, not an

implementation convenience – the frontend must never attempt to run model inference client-side, and must always call the backend for predictions, metrics, or alerts.

- **Local-Only Deployment (New in v1.2):** Both tiers run on `localhost` for this course deliverable (e.g., frontend on port 3000, backend on port 8000); no public hosting or exposure to the internet is in scope.

2.6 User Documentation

- **README.md:** Environment setup for *both* tiers (Python virtual environment for the backend; `npm install / npm run dev` for the Next.js frontend), dataset placement, and execution instructions, including the order in which the two processes must be started.
- **Dashboard Help:** Embedded tooltips (rendered as animated popovers) explaining ML metrics (e.g., F1-score, concept drift) for users without a data-science background.

2.7 Assumptions and Dependencies

- The CICEVSE2024 network-traffic subset is a reasonable proxy for real-world EV-charging network behaviour (OCPP/ISO 15118-adjacent traffic on the lab testbed described in [Buedi et al., 2024]); it is *not* claimed to be representative of every deployed EVCS vendor or protocol version.
- The user has downloaded and extracted the dataset locally before execution; the dataset’s license/terms of use (CIC academic-use terms) are followed.
- Benchmark figures quoted in this document from [Makhmudov et al., 2025] (Appendix E) are literature targets for sanity-checking our own pipeline, not guarantees of what this specific implementation will reproduce.
- **(New in v1.2)** Both the frontend and backend processes are assumed to run on the same machine (or same trusted local network) during evaluation/demonstration, so cross-origin requests are permitted between `localhost:3000` and `localhost:8000` without production-grade authentication; this assumption must be revisited before any deployment beyond a local demo (see SEC-3).

2.8 Positioning vs. Related Work

This subsection is added so the project’s contribution is explicit – required if the work is later written up as a paper or article.

- **Vs. Makhmudov et al. (2025):** That paper is the primary methodological template for the ML core of this project (ARF + ADWIN online IDS on CICEVSE2024’s network-traffic subset, reporting 0.9913 binary accuracy / 0.9956 F1, and 0.9840 multiclass accuracy / 0.9831 F1; see Appendix E). This project reproduces and benchmarks that approach as a course exercise and adds a decoupled, interactive/animated demonstration dashboard, which the original paper does not provide.

- **Vs. Joglekar et al. (2026):** That paper argues a *hybrid* IDS (fusing network, host, and power-consumption signals) improves detection of host-based attacks (e.g., cryptojacking, backdoor) invisible at the network layer alone. This project does not implement that fusion; it is recorded as future work, not silently dropped.
- **Vs. Bean & Manias (2026):** That survey frames open challenges in EVCS cybersecurity, including dataset scarcity, the need for lightweight/edge-deployable models, and adversarial robustness. The lightweight classical-ML backend is consistent with that direction; the new Next.js frontend does not change the ML methodology, only how results are presented.
- **Contribution framing for a write-up:** the publishable angle remains a *reproducibility and comparative-benchmarking study* of static vs. online classifiers on CICEVSE2024’s network dimension; the Next.js dashboard is an engineering/HCI contribution (demonstrability, interactivity) and should be described as such, not conflated with the ML contribution, if written up.

3 External Interface Requirements

3.1 User Interfaces

All items in this subsection are revised in v1.2: the UI is now a Next.js/React single-page application rather than a server-rendered Streamlit app, and acceptance criteria are restated in terms of client-side state updates driven by API/WebSocket events instead of a Streamlit script rerun.

- **UI-1: Main Dashboard.** Built with Next.js (React, TypeScript) and Tailwind CSS. Displays system status, active dataset details, and total processed records, fetched from the backend’s REST API on load. *Acceptance criterion:* the dashboard’s initial data fetch and first render complete within 3 seconds on the reference hardware (Section 2.4).
- **UI-2: Metrics Panel.** Shows updating values for Accuracy, Precision, Recall, and F1-Score per model, animated on change (e.g., value count-up / bar transition via Framer Motion). *Acceptance criterion:* displayed values match the backend’s classification-report JSON for the same model/run to at least 4 decimal places; animated transitions complete within 500 ms of receiving new data.
- **UI-3: Confusion Matrix.** Interactive, animated visualization (e.g., cell highlight/transition on hover or on update) of False Positives vs. True Positives across attack classes, rendered with the chosen charting library. *Acceptance criterion:* matrix cell values sum to the total evaluated instance count; re-render on new data does not exceed one animation frame budget (see PR-2).
- **UI-4: Alert Feed.** A live, animated list (new alerts enter with a transition, e.g., slide/fade-in) driven by the WebSocket alert stream, e.g. `[ALERT] SYN_Flood detected at 15:05:32`. *Acceptance criterion:* every record the backend labels “attack” during a run appears in the feed within 500 ms of the corresponding WebSocket message being sent.

- **UI-5 (New): Simulation/Playback Control.** Since the dashboard now supports “interactive animation simulation,” the frontend shall provide play/pause/speed controls for replaying a labelled traffic stream against the model. *Acceptance criterion:* pausing halts new alert/metric updates within one animation frame; resuming continues from the paused offset, not from the start.

3.2 Hardware Interfaces

- **HI-1:** None. The system operates on standard PC hardware; it does not interface with EVSE hardware, chargers, or vehicles.

3.3 Software Interfaces

- **SI-1: File System I/O.** The backend reads local CSV files (potentially multiple GB) efficiently, e.g. via chunked reads.
- **SI-2: Python ML Libraries.** scikit-learn for static models; River for the adaptive online model and ADWIN.
- **SI-3 (New): Backend API Layer.** The backend shall expose its functionality to the frontend via a documented API, e.g. FastAPI, with:
 - REST endpoints such as `GET /api/status`, `GET /api/metrics`, `GET /api/dataset`, `POST /api/simulate/start`, `POST /api/simulate/pause`.
 - A WebSocket endpoint (e.g. `/ws/alerts`) that pushes one JSON message per flagged instance during a simulation run.
 - All response payloads following a documented JSON schema (field names, types) so the frontend and backend can be developed and tested independently
 - this schema is the new integration contract that did not exist in the v1.1 monolithic design.
- **SI-4 (New): Frontend Libraries.** React 18+, Next.js 14+, TypeScript, Tailwind CSS, Framer Motion (animation), and one charting library (Recharts or visx/D3.js) as selected by the team; the choice should be recorded in the README once fixed.

3.4 Communications Interfaces

- **CI-1: Frontend Web Server (Revised).** The Next.js application serves the browser UI on `localhost:3000` (development server, e.g. `next dev`, or a production build via `next start`).
- **CI-2 (New): Backend API Server.** The Python backend (e.g. FastAPI + Uvicorn) serves REST and WebSocket traffic on `localhost:8000`. The frontend communicates with the backend exclusively over HTTP (REST, JSON) and WebSocket (for the alert stream); *no direct database or file-system access from the frontend is permitted.*
- **CI-3 (New): Cross-Origin Configuration.** Because the frontend (port 3000) and backend (port 8000) are different origins, the backend shall enable CORS for

the frontend’s local origin only, during development/demonstration (see SEC-3). No external network egress occurs from either tier.

4 System Features

4.1 Data Preprocessing & Feature Engineering Pipeline

4.1.1 Description and Priority

Converts raw EV network-traffic logs into standardized, scaled numerical arrays suitable for machine learning. **Priority: High.**

4.1.2 Stimulus/Response Sequences

- **Stimulus:** User specifies the dataset path and initiates the pipeline (now triggered via a REST call from the dashboard, or a backend CLI script).
- **Response:** System drops duplicates, encodes categorical variables (e.g., protocol), applies a robust scaler to numeric features, and outputs `X_train`, `X_test`, `y_train`, `y_test`.

4.1.3 Functional Requirements

- **REQ-1:** The system shall drop features with zero variance or $> 80\%$ missing data. *Acceptance criterion:* post-preprocessing feature report lists zero such features remaining; the dropped-feature list is logged and exposed via `GET /api/status`.
- **REQ-2:** The system shall remove or mask personally identifiable / network-specific fields (e.g., IP addresses, MAC addresses) to prevent the model overfitting to specific hardware identity rather than behaviour. *Acceptance criterion:* a column-name audit confirms no raw IP/MAC field reaches the training matrix or the API response payloads.

4.2 ML Detection Engine (Static & Online)

4.2.1 Description and Priority

The core intelligence layer that classifies flows as Normal or Attack, implementing both static batch models and an adaptive streaming model. **Priority: High.** (*Unchanged by the frontend migration – this stays entirely in the Python backend.*)

4.2.2 Functional Requirements

- **REQ-3:** The system shall implement Random Forest, SVM, Logistic Regression, and Decision Tree classifiers via scikit-learn. *Acceptance criterion:* all four models train without error and each produces a full classification report, retrievable via `GET /api/metrics`.
- **REQ-4:** The system shall implement an Adaptive Random Forest (ARF) with ADWIN drift detection, replicating [Makhmudov et al., 2025; Gomes et al., 2017]. *Acceptance criterion:* the system detects and logs at least one induced drift

event during a drift-injection test, and pushes a drift-event notification over the `/ws/alerts` channel so the frontend can render it distinctly from a normal attack alert.

4.3 Attack Coverage Evaluation Module

4.3.1 Description and Priority

Evaluates the detection engine against (a) the attack traffic already captured within CICEVSE2024, and (b) a controlled, clearly-labelled synthetic traffic generator used only to probe robustness beyond the recorded attack set. **Priority: Medium.** (*Unchanged by the frontend migration.*)

4.3.2 Functional Requirements

- **REQ-5:** The system shall be evaluated first against the real, labelled network-based attack records native to CICEVSE2024. *Acceptance criterion:* the evaluation report, retrievable via the API, states per-attack-category sample counts and recall.
- **REQ-6:** The system shall support programmatic duplication/mutation of existing benign records to simulate flooding-style traffic, for supplementary robustness testing only. *Acceptance criterion:* synthetic records are tagged `synthetic=True` in both the backend data and any API payload, and the frontend visually distinguishes synthetic-derived alerts (e.g., a distinct badge/colour) from native-dataset alerts.
- **REQ-7:** The system shall allow the user to define, via the dashboard’s simulation controls (UI-5) or an API call (`POST /api/simulate/start`), the percentage of synthetic attack traffic injected into a supplementary evaluation stream (configurable, e.g. 5%–50%). *Acceptance criterion:* the injected proportion in the output stream matches the configured value within $\pm 1\%$.

4.4 Alerting and Visualization Dashboard (Revised in v1.2)

4.4.1 Description and Priority

An interactive, animated Next.js/React interface for security analysts, consuming the backend API/WebSocket rather than being rendered server-side by the ML process itself. **Priority: High.**

4.4.2 Functional Requirements

- **REQ-8 :** The system shall push an updating classification report to the dashboard over the API/WebSocket during a simulated stream run. *Acceptance criterion:* the frontend’s displayed report updates within 500 ms of a new backend computation, without a full page reload (single-page-application behaviour).
- **REQ-9:** The system shall log flagged anomalous traffic to a downloadable CSV file, including timestamp, predicted class, model used, and confidence score, and shall expose a “Download CSV” action in the dashboard that calls a corresponding

API endpoint. *Acceptance criterion:* exported CSV row count equals the number of alerts shown in the dashboard for that session.

- **REQ-13 (New): Animated Alert/State Transitions.** The dashboard shall animate (a) new alerts entering the alert feed, (b) metric-value changes in the metrics panel, and (c) confusion-matrix updates, to support the “interactive animation simulation” requirement. *Acceptance criterion:* animations run at a target of ≥ 30 fps on the reference hardware (Section 2.4) and never block receipt of the next WebSocket message (i.e., animation is purely a rendering concern, never gates data ingestion).
- **REQ-14 (New): API Contract Stability.** The backend shall version its API (e.g., `/api/v1/...`) and document the JSON schema for every endpoint and WebSocket message. *Acceptance criterion:* a schema/contract document (e.g., OpenAPI spec generated by FastAPI) exists and matches the actual responses, checked by an automated contract test.

4.5 Evaluation & Reproducibility Requirements

4.5.1 Description and Priority

Priority: High. (*REQ-11’s export format is now also the payload shape for `GET /api/metrics`, so the two are guaranteed consistent.*)

4.5.2 Functional Requirements

- **REQ-10:** The system shall use a fixed, documented train/test split (or stratified k-fold cross-validation, $k \geq 5$) with a recorded random seed. *Acceptance criterion:* re-running the pipeline with the same seed reproduces identical reported metrics to at least 4 decimal places.
- **REQ-11:** The system shall report, for every model, Accuracy, Precision, Recall, F1-Score, and per-instance inference latency, alongside the confusion matrix. *Acceptance criterion:* a single exported results table (CSV/Markdown) and the `GET /api/metrics` JSON response both contain all five metrics for all five models, in matching values.
- **REQ-12:** The system shall persist trained models (e.g., via `joblib`) so that a reported result can be reloaded and re-scored without retraining, and so the API can serve inference without retraining on every request. *Acceptance criterion:* a reloaded model reproduces the same predictions on a held-out sample as the original in-memory model, and API responses after a backend restart are unchanged.

5 Other Nonfunctional Requirements

5.1 Performance Requirements

- **PR-1:** The online model (ARF) shall predict each instance in under 5 milliseconds, consistent with the “lightweight, edge-feasible” direction identified in [Bean

& Manias, 2026]. (*Unchanged – this is a backend-only measurement, independent of the UI.*)

- **PR-2 (Revised):** The Next.js dashboard shall remain responsive to user input (e.g., pausing a simulation, hovering a chart) at all times; the browser’s main thread shall not be blocked for more than 100 ms by any single animation or data-update handler, consistent with standard web-performance guidance for perceived responsiveness.
- **PR-3 (New): Animation Frame Budget.** All chart/alert animations (REQ-13) shall target 30–60 fps and shall degrade gracefully (e.g., reduce animation complexity, not drop data) if the reference hardware cannot sustain that rate.
- **PR-4 (New): API Latency.** REST endpoints used for dashboard polling/initial load shall respond within 300 ms on the reference hardware over localhost; WebSocket alert messages shall be delivered to the frontend within 200 ms of the backend generating them.

5.2 Safety Requirements

- **SAF-1:** The system shall not execute active penetration testing or send packets outward over actual network interfaces. All testing is local, offline, and simulated – no live EVSE, CSMS, or grid system is contacted at any point. (*Unchanged.*)

5.3 Security Requirements

- **SEC-1:** System logs and outputs shall not retain identifiable PII (IP/MAC addresses) from the original network captures, consistent with REQ-2; this now also applies to every API response and WebSocket message.
- **SEC-2:** The dataset copy used for training shall not be redistributed by this project; only aggregate results and derived features may be shared.
- **SEC-3 (New):** Because the frontend and backend are now separate network-addressable processes (even if both are on `localhost`), the backend’s CORS policy (CI-3) shall restrict allowed origins to the known local frontend origin(s) rather than using a wildcard (*), and the API shall not be bound to a non-loopback interface for this project’s scope (see Section 2.5, Local-Only Deployment).

5.4 Software Quality Attributes

- **Reliability:** The system must handle class imbalance (attack classes are a minority in CICEVSE2024) gracefully, without crashing or silently reporting misleading accuracy on the majority class alone.
- **Maintainability (Revised):** The system is now three cleanly decoupled modules: (1) the ML pipeline (Python: data, features, models), (2) the API layer (FastAPI), and (3) the presentation layer (Next.js/React/TypeScript). This is a stronger separation of concerns than the v1.1 design, where the UI and ML runtime were one Streamlit process; it also means the frontend and backend can now be developed, tested, and even graded/demonstrated somewhat independently.

- **Reproducibility:** See REQ-10–REQ-12; unchanged in substance, now also guarantees the API and the offline report never disagree.
- **Portability (New):** The frontend, being a standard Next.js app, can later be containerized or statically exported independently of the Python backend, which is useful if the team later wants to demo the UI without the full ML environment installed (e.g., using recorded/mock API responses).

5.5 Business Rules

- **BR-1:** The system relies purely on open-source libraries and open-access data, maintaining zero cost for deployment and academic evaluation. (*Next.js, React, Tailwind CSS, Framer Motion, Recharts/visx, and FastAPI are all open-source/MIT-licensed, so this rule still holds.*)

6 Other Requirements

- **OR-1:** The project must satisfy the academic requirements for the Software Engineering course, including code modularity and documentation standards.
- **OR-2:** If results are submitted for publication or a departmental journal, the abstract/introduction must state the contribution honestly as a reproducibility/benchmarking study (Section 2.9), and must cite Buedi et al. (2024) as the dataset source in addition to any paper whose method is replicated. The dashboard/animation work, if mentioned, should be framed as a demonstration/HCI contribution, not as a novel detection method.

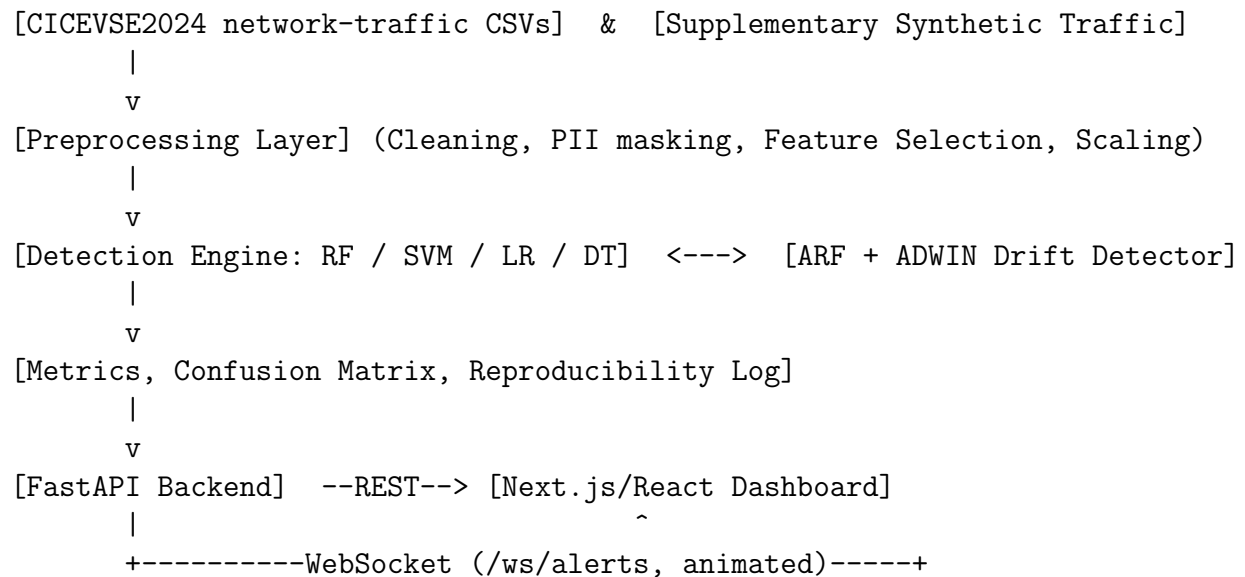
A Appendix A: Glossary

- **EVCS:** Electric Vehicle Charging Station.
- **EVSE:** Electric Vehicle Supply Equipment (the physical charger). CICEVSE2024 uses two: EVSE-A (a real charger) and EVSE-B (emulated on a Raspberry Pi).
- **CSMS:** Charging Station Management System (the cloud backend).
- **OCPP:** Open Charge Point Protocol; standard protocol between EVSE and CSMS.
- **IDS:** Intrusion Detection System.
- **ADWIN:** ADaptive WINdowing; an algorithm used to detect concept drift in data streams.
- **ARF:** Adaptive Random Forest; a machine-learning algorithm suited to online streaming data.
- **Concept Drift:** A phenomenon where the statistical properties of the target variable change over time, requiring models to adapt.
- **SSR:** Server-Side Rendering, a Next.js rendering mode (not required for this project, which primarily needs client-side rendering for live/animated updates).
- **WS:** WebSocket, a persistent, bidirectional connection used here to push live alerts from the backend to the frontend.
- **CORS:** Cross-Origin Resource Sharing; the browser security mechanism that must be configured (CI-3) because the frontend and backend now run on different ports/origins.

B Appendix B: Analysis Models

Note: below is a text representation of the system data flow; in the final report this would be replaced with a rendered DFD/UML diagram.

Level 0 Data Flow (Revised for v1.2):



C Appendix C: To Be Determined List

- **TBD-1:** Exact number of features to retain post Recursive Feature Elimination (RFE).
- **TBD-2 :** Whether the Next.js frontend is deployed (e.g., to Vercel) for demonstration purposes while the Python backend remains local-only (in which case the deployed frontend would need a tunnelled or otherwise reachable backend URL), versus running both tiers strictly locally for grading. (*Superseded the v1.1 “Streamlit Community Cloud” question, which no longer applies.*)
- **TBD-3:** Whether the team pursues the host-event/power-consumption fusion extension (Section 2.9) as a stretch goal before any external submission.
- **TBD-4 (New):** Choice between Recharts (simpler, less flexible) and visx/D3.js (more flexible, steeper learning curve) for the animated charts in UI-2/UI-3 – to be decided once the team has prototyped both against REQ-13’s frame-rate target.
- **TBD-5 (New):** Whether WebSocket is implemented via FastAPI’s native `WebSocket` support or via `python-socketio` (which pairs with `socket.io-client` on the frontend and offers automatic reconnection) – affects SI-3/SI-4 and CI-2.

D Appendix D: Requirements Traceability Matrix

ID	Requirement (summary)	Source	Verification Method	Priority
REQ-1	Drop low-value features	Course brief	Feature audit log	High
REQ-2	Mask PII (IP/MAC), incl. API payloads	Course brief; privacy best practice	Column & payload audit	High
REQ-3	Static classifiers (RF/SVM/LR/DT)	Course brief; benchmark vs. Makhmudov et al. 2025	Classification report via API	High
REQ-4	ARF + ADWIN online model	Makhmudov et al. 2025; Gomes et al. 2017	Drift-injection test + WS event	High
REQ-5	Evaluate on native CI-CEVSE2024 attacks	Buedi et al. 2024	Per-class recall report	Medium
REQ-6	Tag synthetic vs. native samples in data & API	Data-integrity correction (v1.1)	Flag audit (backend + frontend)	Medium
REQ-7	Configurable synthetic injection ratio via API/UI-5	Course brief	Ratio-tolerance test	Medium

ID	Requirement (summary)	Source	Verification Method	Priority
REQ-8	Live-updating classification report over API/WS	Course brief; revised for Next.js (v1.2)	UI + WS latency test	High
REQ-9	CSV alert export via API-backed dashboard action	Course brief	Row-count cross-check	High
REQ-10	Fixed seed / cross-validation	Reproducibility (v1.1)	Repeat-run comparison	High
REQ-11	Full 5-metric results table (export + API)	Reproducibility (v1.1)	Export/API consistency check	High
REQ-12	Model persistence (joblib), serves API	Reproducibility (v1.1); API serving (v1.2)	Reload-and-compare test	Medium
REQ-13	Animated alert/metric/matrix transitions	New in v1.2 (interactive simulation requirement)	Frame-rate benchmark	High
REQ-14	Versioned, documented API contract	New in v1.2 (frontend/backend decoupling)	OpenAPI/contract test	High
PR-1	<5 ms inference latency	Bean & Manias 2026 (edge feasibility)	Timed benchmark	High
PR-2	UI thread not blocked >100 ms	Revised in v1.2 (Next.js/React)	Performance profiling	High
PR-3	30–60 fps animation budget	New in v1.2	Frame-rate benchmark	Medium
PR-4	REST <300 ms, WS <200 ms	New in v1.2	Timed API/WS test	High
SAF-1	No live network egress	Course brief	Code/network audit	High
SEC-3	CORS restricted to known local origin	New in v1.2 (two-process architecture)	Backend config review	High

E Appendix E: Literature Benchmark Reference Table

These figures are reported results from [Makhmudov et al., 2025] on CICEVSE2024, included so this team’s own results can be compared honestly against a published baseline using the same dataset. They are **targets for comparison, not claims about this project’s own implementation**; they are unaffected by the v1.2 frontend migration,

since the ML methodology is unchanged.

Task	Accuracy	Precision	Recall	F1-Score
Binary classification	0.9913	0.9999	0.9914	0.9956
Multiclass classification	0.9840	0.9840	0.9840	0.9831

Table 2: Reported ARF+ADWIN performance on CICEVSE2024 [Makhmudov et al., 2025]. Average accuracy during drift events: ≈ 0.99 (binary), ≈ 0.96 (multiclass).